开源小答

课程知识库智能问答助手

基于 LangChain 框架与检索增强生成（RAG）的智能应用

《开源软件》课程 · 第七章 智能应用开发 · 期末作业

项目名称	开源小答 · 课程知识库智能问答助手
开源仓库	https://github.com/Yanghanrui666/-
技术栈	Python 3.12 · LangChain 0.3 · Streamlit · Ollama
开源许可证	MIT License
文档版本	v1.0

目　录

一、项目简介.....3
二、功能特性.....3
三、系统架构.....4
四、技术实现.....5
五、运行与配置.....6
六、使用说明.....7
七、测试与验证.....8
八、项目目录结构.....9
九、常见问题.....9
十、许可证与致谢.....10

文档说明

本文档为智能应用「开源小答」的 README 文档说明，共 10 章，包含 1 幅系统架构图、8 张参数与配置表、5 段示例代码与命令。内容覆盖项目定位、技术实现、运行配置、使用方法与测试验证，可作为仓库 README 的完整离线版本提交。

对应的开源仓库地址见封面与附录，仓库内 README.md 与本文档内容一致。若需复现文中结果，请按第五章的步骤安装依赖并启动应用；不配置 API Key 时应用默认运行在演示模式。

一、项目简介

「开源小答」是一款面向课程学习的智能问答助手。它把课程资料（开源许可证、Git 协作流程、LangChain 框架、检索增强生成等）作为本地知识库，用户提问时先从知识库中检索相关片段，再交由大模型基于这些片段生成回答，并在答案中标注来源。

与传统“直接询问大模型”的方式相比，该应用具有三点差异：

- 答案有据可依：模型只依据检索到的资料作答，资料不足时明确说明，降低幻觉风险。
- 结果可溯源：每个回答附带来源文件名与片段编号，便于人工核对。
- 知识可更新：只需向 data/knowledge 目录放入新的 Markdown 或 TXT 文件，无需重新训练模型。

1.1 设计目标

目标	具体做法
可复现	克隆仓库后无需任何 API Key 即可跑通全流程（内置演示模式）
可切换	同一套代码支持在线模型、本地离线模型与规则模型，只改一个环境变量
可解释	回答附带检索来源与片段编号，界面可展开查看原文

1.2 应用形态

应用提供两种交互形态：一是基于 Streamlit 的 Web 聊天界面，支持多轮对话、参数实时调节与命中片段查看；二是命令行工具，支持单轮提问与交互式会话，便于脚本化验证与无图形环境使用。

二、功能特性

功能模块	说明
RAG 问答	文档加载 → 递归切块 → 检索 → 提示词拼装 → 大模型生成 → 来源溯源
三种模型模式	demo（离线规则模型）/ openai（OpenAI 兼容接口）/ ollama（本地离线模型）
双检索后端	默认零依赖 TF-IDF（中文单字 + 二元组切分）；可切换向量检索
多轮对话	历史消息通过 MessagesPlaceholder 注入提示词，支持上下文追问
Web 界面	Streamlit 聊天界面，参数实时可调，命中片段可展开查看
命令行	单轮提问与交互式会话，适合脚本化与快速验证
可测试	演示模型使端到端链路可在无网络环境下被 pytest 覆盖

三、系统架构

3.1 总体架构

系统自上而下分为四层：交互层（Web 界面与命令行）、编排层（LCEL 问答链）、能力层（模型接入与检索）、配置层（.env 解析）。依赖方向单向，交互层不直接接触模型 SDK，替换模型或检索实现不影响界面。

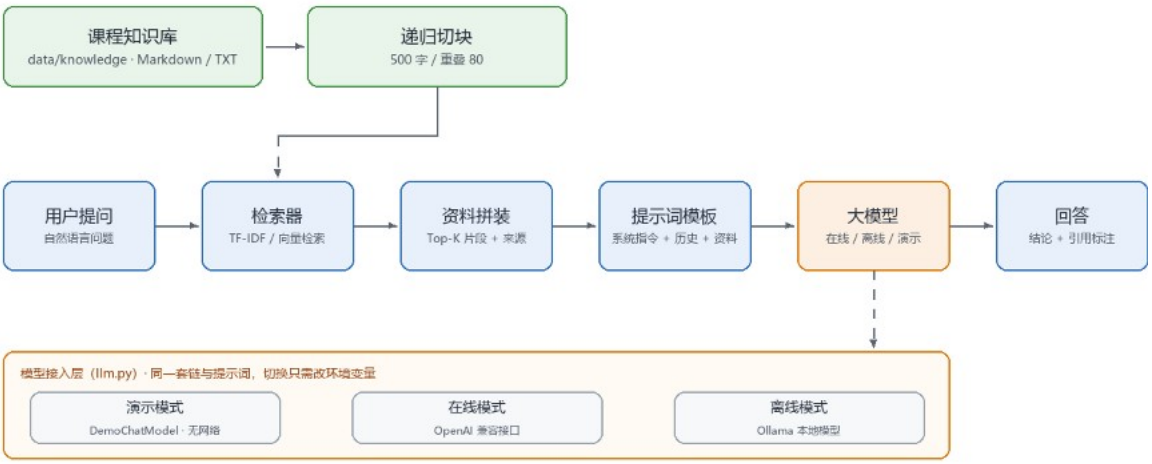


图 1　系统总体架构与请求流转路径

3.2 核心问答链

问答流程使用 LangChain 表达式语言（LCEL）组合为一个 Runnable，逐级追加字段而不覆盖原始输入，因此调试时可以直接打印任一阶段的中间结果。

```
chain = (
    RunnablePassthrough.assign(documents=retrieve)
    | RunnablePassthrough.assign(context=format_documents)
    | RunnablePassthrough.assign(answer=prompt | llm | StrOutputParser())
    | RunnableLambda(lambda x: Answer(text=x["answer"], sources=x["documents"]))
)
answer = chain.invoke({"question": "GPL 和 MIT 有什么区别？", "history": []})
```

3.3 数据传递

链的输入是一个字典，随流程依次增加 documents（检索结果）、context（拼装后的参考资料）、answer（模型回答）三个字段，最后转换为 Answer 数据类，携带答案文本与来源文档列表返回给交互层。

四、技术实现

4.1 文档加载与切块

使用 RecursiveCharacterTextSplitter 按“段落 → 换行 → 句号 → 逗号”的优先级递归切分，默认块大小 500 字、重叠 80 字。每个片段保留来源文件名与块序号，写入 metadata，供回答时溯源。

4.2 中文检索

默认检索器为纯 Python 实现的 TF-IDF 余弦相似度，不依赖任何 embedding 模型，因此完全离线可用。针对中文，分词采用“单字 + 相邻二元组”策略：

- 英文与数字按词整体切分，例如 langchain、3.12。
- 中文连续字符先按单字切分，再补充相邻二元组，使“开源许可证”能命中“许可证”等子串。
- IDF 由语料统计得出，查询与片段分别计算 TF-IDF 向量后取余弦相似度排序。

当配置了 API Key 并开启 SMARTQA_USE_EMBEDDINGS 时，检索后端切换为 InMemoryVectorStore + embedding 接口，接口签名保持一致，上层代码无需修改。

4.3 提示词设计

规则	作用
只依据参考资料作答	把模型的自由度约束在检索结果内，抑制幻觉
资料不足时明确说明	避免模型用常识“补全”，提高可信度
先结论后要点	符合教学场景的阅读习惯，便于快速抓取重点
标注来源编号	支持人工核对，也为后续评估提供依据

4.4 三种模型接入

模式	实现类	特点	适用场景
demo	DemoChatModel	规则抽取，零依赖、无网络	本地自检、单元测试、无 Key 演示
openai	ChatOpenAI	自定义 base_url，兼容多家厂商	DeepSeek / 通义 / Kimi / OpenAI
ollama	ChatOllama	本地量化模型，数据不出本机	完全离线、隐私敏感场景

三种模式共享同一条链、同一套提示词与同一个检索器，因此切换模型不会改变应用行为，这也使得自动化测试可以在无网络环境下覆盖完整链路。

4.5 多轮对话

LangChain 0.3 推荐由应用层管理消息历史。界面把每轮的用户消息与模型消息存入会话状态，提问时整体装入 MessagesPlaceholder，既保留上下文，又便于在界面上清空或截断历史。

五、运行与配置

5.1 环境要求

项目	要求
Python	3.10 及以上（本项目在 3.12 验证）
操作系统	Windows / macOS / Linux
硬件	无需 GPU；演示模式完全离线

5.2 安装

```
git clone https://github.com/Yanghanrui666/-.git smartqa
cd smartqa

python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate        # macOS / Linux

pip install -r requirements.txt
```

5.3 配置项

环境变量	默认值	说明
SMARTQA_PROVIDER	demo	模型提供方：demo / openai / ollama
SMARTQA_MODEL	deepseek-chat	模型名称
SMARTQA_BASE_URL	https://api.deepseek.com/v1	OpenAI 兼容接口或 Ollama 地址
SMARTQA_API_KEY	（空）	在线模式必填；不会写入仓库
SMARTQA_TEMPERATURE	0.2	生成温度，越低越稳定
SMARTQA_TOP_K	4	每次检索的片段数
SMARTQA_KNOWLEDGE_DIR	./data/knowledge	知识库目录
SMARTQA_USE_EMBEDDINGS	false	是否启用向量检索

5.4 启动

```
# Web 界面
streamlit run app.py

# 命令行：单轮问答
python -m smartqa.cli -q "GPL 和 MIT 许可证有什么区别？"

# 命令行：交互式多轮会话
python -m smartqa.cli -i --show-sources
```

六、使用说明

6.1 Web 界面

浏览器打开终端提示的地址（默认 http://localhost:8501）。左侧栏可切换模型提供方、填写 API Key、调节温度与检索片段数、指定知识库目录；主区域为对话窗口，回答下方的“参考资料”折叠面板可展开查看命中的原文片段。

6.2 命令行示例

在演示模式下执行一条真实提问，实际输出如下（节选）：

```
$ python -m smartqa.cli -q "GPL 和 MIT 许可证有什么区别？" --show-sources
模型：演示模式（无需网络，规则模型） | 知识片段：12

（演示模式回答）关于「GPL 和 MIT 许可证有什么区别？」，知识库中的相关信息如下：

- **许可证兼容性**：把 GPL 代码并入 MIT 项目会迫使整个项目遵循 GPL；
- | MIT | 宽松 | 是 | 否 | 是 |
- | LGPL-3.0 | 弱 copyleft | 是 | 动态链接可闭源 | 是 |

--- 参考来源 ---
  · 01-开源许可证.md #0
  · 01-开源许可证.md #1
  · 02-Git 与协作流程.md #3
```

演示模式的回答由规则模型根据检索结果整理，用于验证“检索 → 提示词 → 解析”整条链路；配置真实模型后，同样的命令会返回大模型生成的完整回答。

七、测试与验证

测试全部使用演示模型，不依赖网络与 API Key，可直接在 CI 中运行。执行 pytest 即可复现下表结果。

测试范围	覆盖内容	数量
配置解析	默认值、环境变量覆盖、非法值回落、不可变替换	4
检索层	中文分词、文档加载、切块元数据、TF-IDF 排序、空库与无重叠兜底	7
问答链	模型构造、缺 Key 校验、索引构建、回答与来源、多轮历史、空问题校验	8
合计	全部通过（pytest，19 passed）	19

除自动化测试外，还进行了两项手工验证：其一，在无网络环境启动 Streamlit 服务，确认首页返回 200 且健康检查为 ok；其二，在命令行执行真实提问，确认回答内容与来源标注正确。

八、项目目录结构

```
.
├── app.py                      # Streamlit Web 界面
├── src/smartqa/
│   ├── __init__.py             # 公开 API
│   ├── config.py               # .env 配置解析
│   ├── llm.py                  # 三种模型接入 + 演示模型
│   ├── retriever.py            # 文档加载、切块、TF-IDF / 向量检索
│   ├── chain.py                # 提示词与 LCEL 问答链
│   └── cli.py                  # 命令行入口
├── data/knowledge/             # 知识库（4 篇课程资料）
├── tests/                      # pytest 单元与端到端测试
├── docs/architecture.md        # 架构说明
├── requirements.txt            # 运行依赖
├── pyproject.toml              # 包元数据与 pytest 配置
├── .env.example                # 配置模板
└── LICENSE                     # MIT
```

九、常见问题

Q：启动报 API Key 相关错误?

A：在线模式必须设置 SMARTQA_API_KEY；若只想先看效果，把 SMARTQA_PROVIDER 改回 demo。

Q：Ollama 连接失败?

A：确认 ollama serve 正在运行，SMARTQA_BASE_URL 与端口一致（默认 11434），且模型已 pull。

Q：回答提示“资料中没有找到相关内容”?

A：说明检索未命中。可换用文档中的术语提问、调大 SMARTQA_TOP_K，或向知识库补充资料。

Q：如何换成自己的知识库?

A：把 Markdown / TXT 文件放入 data/knowledge，或修改 SMARTQA_KNOWLEDGE_DIR，界面会自动重新索引。

Q：为什么默认不用向量检索?

A：为了“克隆即可运行”。开启 SMARTQA_USE_EMBEDDINGS 并提供 API Key 后即切换为向量检索。

十、许可证与致谢

本项目采用 MIT License 开源，允许自由使用、修改与再分发，只需保留版权声明与许可声明。完整许可证文本见仓库根目录 LICENSE 文件。

致谢以下开源项目：

- LangChain —— 大模型应用编排框架，提供 Runnable、Prompt、Retriever 等核心抽象。
- Streamlit —— 快速构建数据与 AI 应用界面。
- Ollama —— 本地大模型运行时，使应用可以完全离线运行。

附录　仓库信息

项目	内容
开源仓库	https://github.com/Yanghanrui666/-
分支	main
许可证	MIT
运行方式	streamlit run app.py / python -m smartqa.cli